



Module 08 — Jointures : relier les tables

Bootcamp Data Analyst — From Zero to Hero | Niveau Débutant · Acte III



Ce que tu seras capable de faire à la fin de ce module

- Comprendre le modèle relationnel : clés primaires, clés étrangères
- Utiliser les alias de table pour écrire des requêtes lisibles
- Relier deux ou trois tables avec `INNER JOIN`
- Garder les lignes sans correspondance avec `LEFT JOIN`
- Détecter les orphelins avec l'anti-join (`WHERE x IS NULL`)
- Simuler un `FULL OUTER JOIN` en SQLite avec `UNION ALL`
- Combiner jointures et agrégations pour répondre à des questions business



Durée estimée : 45 minutes



Prérequis : Modules 06 et 07 — tu maîtrises `SELECT`, `WHERE`, `GROUP BY`, `HAVING`



Environnement : sqliteonline.com — zéro installation

1. Pourquoi les jointures ?

Au Module 07, tu as appris à résumer des données avec `GROUP BY`. Mais toutes les infos dont tu as besoin ne sont jamais dans **une seule table**.

Exemple réel : ton DG te demande un rapport clients.

Ta table `transactions` contient les montants — mais **pas les noms des clients**.

Ta table `clients` contient les noms — mais **pas les montants**.

transactions		clients
id_client = 1042	?→	nom = "Kouassi Ama"
montant = 2 000		ville = "Abidjan"

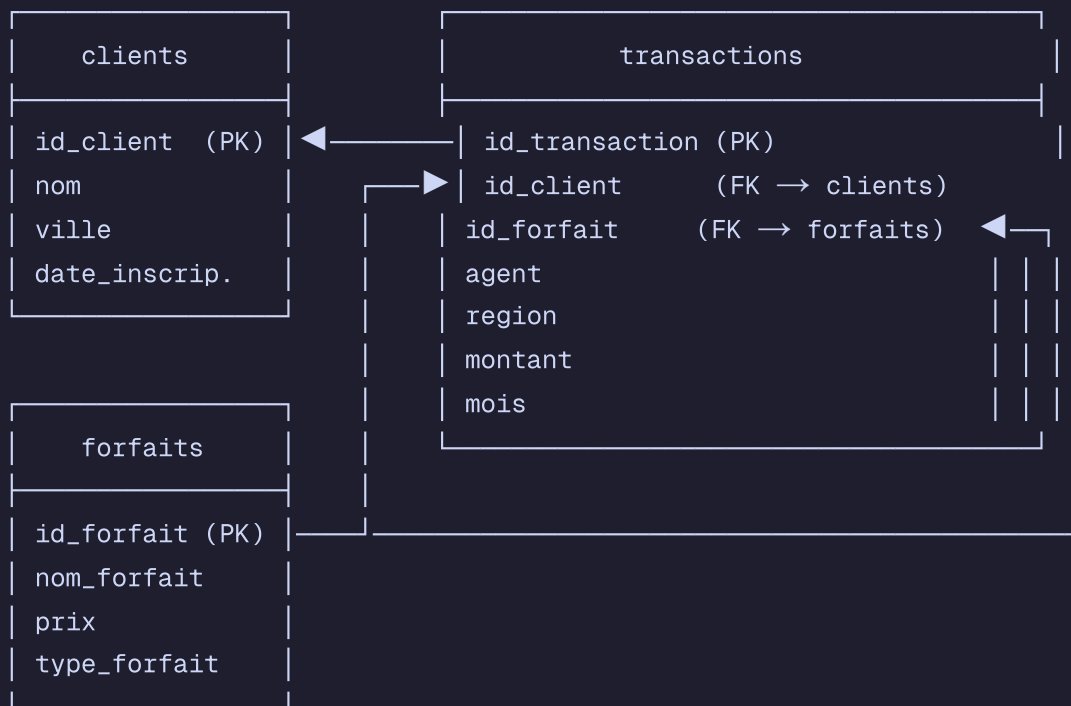
Sans jointure, tu ne peux pas répondre à *"Qui a dépensé combien ?"*.

Les jointures, c'est le pont qui relie les tables entre elles.

💡 Dans une vraie base de données, les informations sont volontairement séparées en plusieurs tables pour éviter les répétitions. C'est le principe du modèle relationnel. Les jointures sont le mécanisme pour les reconstituer au moment de l'analyse.

2. Le schéma relationnel — la carte de ce module

On travaille avec **3 tables** d'un opérateur télécom fictif (contexte ANSUT-CI).



PK = Primary Key (clé primaire) · **FK** = Foreign Key (clé étrangère)

Créer les 3 tables dans sqliteonline.com

Colle ce script et exécute-le avant de commencer les exercices :

SQL

```
CREATE TABLE clients (  
  id_client      INTEGER PRIMARY KEY,  
  nom            TEXT,  
  ville         TEXT,  
  date_inscription TEXT  
);
```

```
CREATE TABLE forfaits (  
  id_forfait    INTEGER PRIMARY KEY,  
  nom_forfait   TEXT,  
  prix          INTEGER,  
  type_forfait  TEXT  
);
```

```
CREATE TABLE transactions (  
  id_transaction INTEGER PRIMARY KEY,  
  id_client      INTEGER,  
  id_forfait     INTEGER,  
  agent          TEXT,  
  region         TEXT,  
  montant        INTEGER,  
  mois           TEXT  
);
```

INSERT INTO clients VALUES

```
(1042, 'Kouassi Ama',      'Abidjan',      '2023-06-15'),  
(2317, 'Traoré Bakary',    'Bouaké',       '2023-08-20'),  
(1890, 'Yao Serge',        'Abidjan',      '2023-09-10'),  
(3451, 'Koné Mariama',     'San Pedro',    '2024-01-05'),  
(2100, 'Diallo Ibrahim',   'Abidjan',      '2023-11-22'),  
(1230, 'Ouédraogo Fatima', 'Yamoussoukro', '2024-02-14'),  
(7001, 'Bamba Issa',       'Abidjan',      '2024-03-01'),  
(7002, 'Gnangon Claire',   'Bouaké',       '2024-03-15');
```

INSERT INTO forfaits VALUES

```
(1, 'Recharge_500',      500, 'Recharge'),  
(2, 'Recharge_1000',    1000, 'Recharge'),  
(3, 'Forfait_internet', 2000, 'Forfait'),  
(4, 'Forfait_voix',      1500, 'Forfait'),  
(5, 'Forfait_data_nuit', 800, 'Forfait'),  
(6, 'Pack_famille',      3500, 'Pack');
```

INSERT INTO transactions VALUES

```
(1, 1042, 3, 'Koné Mamadou', 'Abidjan', 2000, '2024-01'),  
(2, 2317, 1, 'Traoré Aminata', 'Bouaké', 500, '2024-01'),
```

```
(3, 1890, 2, 'Kouassi Éric', 'Abidjan', 1000, '2024-01'),
(4, 3451, 4, 'N' 'Guessan Fatou', 'San Pedro', 1500, '2024-01'),
(5, 2100, 1, 'Koné Mamadou', 'Abidjan', 500, '2024-01'),
(6, 1230, 3, 'Bamba Seydou', 'Yamoussoukro', 2000, '2024-02'),
(7, 2317, 3, 'Traoré Aminata', 'Bouaké', 2000, '2024-02'),
(8, 1042, 2, 'Kouassi Éric', 'Abidjan', 1000, '2024-02'),
(9, 3451, 1, 'N' 'Guessan Fatou', 'San Pedro', 500, '2024-02'),
(10, 2100, 4, 'Koné Mamadou', 'Abidjan', 1500, '2024-02'),
(11, 1890, 2, 'Bamba Seydou', 'Yamoussoukro', 1000, '2024-02'),
(12, 1042, 3, 'Kouassi Éric', 'Abidjan', 2000, '2024-03');
```

📌 Observe bien : les clients **7001** (Bamba Issa) et **7002** (Gnangon Claire) n'ont **aucune transaction**. Les forfaits **5** (Forfait_data_nuit) et **6** (Pack_famille) n'ont **jamais été vendus**. Ces lignes seront essentielles pour comprendre LEFT JOIN et l'anti-join.

3. Clés primaires, clés étrangères et alias de table

3.1 Clé primaire (PRIMARY KEY)

La **clé primaire** est une colonne dont la valeur est **unique pour chaque ligne** de la table. Elle permet d'identifier une ligne sans ambiguïté.

TABLE	CLÉ PRIMAIRE	GARANTIE
clients	id_client	Chaque client a un identifiant unique
forfaits	id_forfait	Chaque forfait a un identifiant unique
transactions	id_transaction	Chaque transaction a un identifiant unique

3.2 Clé étrangère (FOREIGN KEY)

La **clé étrangère** est une colonne qui fait référence à la clé primaire d'une autre table. C'est le **pont** entre deux tables.

Dans `transactions` :

- `id_client` → référence `clients.id_client`
- `id_forfait` → référence `forfaits.id_forfait`

💡 La clé étrangère est toujours une valeur qui **existe déjà** dans l'autre table. Si `id_client = 1042` apparaît dans `transactions`, c'est qu'il y a bien un client avec cet ID dans la table `clients`.

3.3 Alias de table — syntaxe indispensable avec JOIN

Quand on joint plusieurs tables, les noms de colonnes peuvent se retrouver en **conflit**. Par exemple, `clients` et `transactions` ont toutes les deux une colonne `id_client`.

Les **alias de table** résolvent ce problème — et rendent les requêtes beaucoup plus lisibles :

```
SQL
-- Sans alias - illisible et ambigu
SELECT clients.nom, transactions.montant
FROM transactions
JOIN clients ON transactions.id_client = clients.id_client;

-- Avec alias - propre et standard
SELECT c.nom, t.montant
FROM transactions t
JOIN clients c ON t.id_client = c.id_client;
```

Convention qu'on utilisera dans tout ce module :

TABLE	ALIAS
transactions	t
clients	c
forfaits	f

⚠ **Règle d'or** : dès qu'une requête JOIN implique plusieurs tables, **utilise toujours les alias**. C'est une convention universelle que tu retrouveras dans tout code SQL professionnel.

3.4 Pont Excel → SQL : RECHERCHEV c'est un JOIN

Si tu viens d'Excel, tu connais déjà l'idée derrière les jointures — sans le savoir.

Scénario Excel classique :

Tu as une feuille `transactions` avec une colonne `id_client`, et une feuille `clients` avec les noms. Pour afficher le nom dans la feuille transactions, tu écris :

```
EXCEL
=RECHERCHEV(A2; clients!$A:$B; 2; FAUX)
```

Ce que fait SQL avec JOIN :

```
SQL
SELECT t.id_transaction, c.nom, t.montant
FROM transactions t
JOIN clients c ON t.id_client = c.id_client;
```

CONCEPT EXCEL	ÉQUIVALENT SQL
<code>RECHERCHEV(cle; table; col; FAUX)</code>	<code>JOIN table ON cle = cle</code>
Valeur cherchée (A2)	<code>ON t.id_client</code>
Plage de la table (clients!\$A:\$B)	<code>clients c</code>
Numéro de colonne retournée	<code>SELECT c.nom</code>

Avantages de SQL sur RECHERCHEV :

- Pas de limite de colonnes — tu récupères autant de colonnes que tu veux
- Pas de colonnes intermédiaires à insérer dans ta feuille
- JOIN peut récupérer des données de 3, 4, 5 tables en une seule requête
- RECHERCHEV retourne la **première** correspondance — JOIN retourne **toutes**

💡 *Un INNER JOIN est exactement un RECHERCHEV qui ne garde que les lignes trouvées. Un LEFT JOIN est un RECHERCHEV qui garde aussi les lignes non trouvées (avec `#N/A` → `NULL` en SQL).*

4. INNER JOIN — seulement les lignes qui matchent

Le principe

`INNER JOIN` retourne uniquement les lignes qui ont **une correspondance dans les deux tables**.

Table A		Table B		INNER JOIN
1042	←→	1042	✓	1042 matchée
2317	←→	2317	✓	2317 matchée
7001	←→	???	✗	absente du résultat
7002	←→	???	✗	absente du résultat

Si un client n'a aucune transaction, il **disparaît** du résultat d'un INNER JOIN.

Syntaxe

```
SQL
SELECT colonnes
FROM table_gauche alias_g
INNER JOIN table_droite alias_d ON alias_g.cle = alias_d.cle;
```

Le mot **INNER** est optionnel — **JOIN** seul fait un INNER JOIN par défaut.

Exemple 1 — 2 tables : transactions + clients

```
SQL
SELECT
  t.id_transaction,
  c.nom,
  c.ville,
  t.montant,
  t.mois
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
ORDER BY t.id_transaction;
```

ID_TRANSACTION	NOM	VILLE	MONTANT	MOIS
1	Kouassi Ama	Abidjan	2 000	2024-01
2	Traoré Bakary	Bouaké	500	2024-01
3	Yao Serge	Abidjan	1 000	2024-01
4	Koné Mariama	San Pedro	1 500	2024-01
5	Diallo Ibrahim	Abidjan	500	2024-01
...	...	...	...	...

→ **12 lignes** — les 12 transactions toutes matchées. Les clients 7001 et 7002 (sans transaction) n'apparaissent pas.

Exemple 2 — 3 tables : transactions + clients + forfaits

SQL

```
SELECT
  c.nom,
  f.nom_forfait,
  f.type_forfait,
  t.montant,
  t.mois
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
JOIN forfaits f ON t.id_forfait = f.id_forfait
ORDER BY t.id_transaction
LIMIT 5;
```

NOM	NOM_FORFAIT	TYPE_FORFAIT	MONTANT	MOIS
Kouassi Ama	Forfait_internet	Forfait	2 000	2024-01
Traoré Bakary	Recharge_500	Recharge	500	2024-01
Yao Serge	Recharge_1000	Recharge	1 000	2024-01
Koné Mariama	Forfait_voix	Forfait	1 500	2024-01
Diallo Ibrahim	Recharge_500	Recharge	500	2024-01

💡 Avec 3 tables et 2 JOIN, on obtient une vue enrichie : on sait qui a acheté quoi. C'est ça, la puissance du modèle relationnel.

5. LEFT JOIN — garder tout le côté gauche

Le principe

LEFT JOIN retourne **toutes les lignes de la table de gauche**, même si elles n'ont pas de correspondance à droite. Les colonnes de droite sont alors **NULL**.

Table gauche		Table droite		LEFT JOIN
1042	↔	1042	✓	1042 · données droite
2317	↔	2317	✓	2317 · données droite
7001	↔	???	✗	7001 · NULL NULL NULL
7002	↔	???	✗	7002 · NULL NULL NULL

La table de **gauche** est celle qui suit **FROM**. La table de **droite** est celle qui suit **LEFT JOIN**.

Syntaxe

SQL

```
SELECT colonnes
FROM table_gauche alias_g
LEFT JOIN table_droite alias_d ON alias_g.cle = alias_d.cle;
```

Exemple — Tous les clients, même ceux sans transaction

SQL

```
SELECT
  c.nom,
  c.ville,
  t.id_transaction,
  t.montant
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client
ORDER BY c.id_client;
```

NOM	VILLE	ID_TRANSACTION	MONTANT
Kouassi Ama	Abidjan	1	2 000
Kouassi Ama	Abidjan	8	1 000
Kouassi Ama	Abidjan	12	2 000
Traoré Bakary	Bouaké	2	500
...	...	...	...
Bamba Issa	Abidjan	NULL	NULL
Gnangon Claire	Bouaké	NULL	NULL

→ **14 lignes** — 12 transactions + 2 clients sans transaction (avec NULL).

5.1 L'anti-join — trouver les orphelins

L'anti-join est une technique qui utilise `LEFT JOIN` + `WHERE x IS NULL` pour trouver les lignes qui n'ont pas de correspondance dans l'autre table.

Question : Quels clients n'ont jamais effectué de transaction ?

SQL

```
SELECT c.nom, c.ville, c.date_inscription
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client
WHERE t.id_transaction IS NULL;
```

NOM	VILLE	DATE_INSCRIPTION
Bamba Issa	Abidjan	2024-03-01
Gnangon Claire	Bouaké	2024-03-15

✅ Le `WHERE` filtre uniquement les lignes où la jointure n'a rien trouvé — c'est-à-dire les clients sans transaction.

Question : Quels forfaits n'ont jamais été vendus ?

SQL

```
SELECT f.nom_forfait, f.prix, f.type_forfait
FROM forfaits f
LEFT JOIN transactions t ON f.id_forfait = t.id_forfait
WHERE t.id_transaction IS NULL;
```

NOM_FORFAIT	PRIX	TYPE_FORFAIT
Forfait_data_nuit	800	Forfait
Pack_famille	3 500	Pack

💡 L'anti-join est l'une des techniques SQL les plus demandées en entretien DA. Retiens le pattern : `LEFT JOIN` + `WHERE cle_droite IS NULL`.

5.2 Exercice pratique — Mets en pratique

Tu vas répondre à 3 questions business en écrivant toi-même les requêtes. Les données sont déjà chargées dans sqliteonline.com depuis la section 2.

Exercice A — Rapport agent

Le directeur veut savoir combien chaque agent a collecté de transactions et de chiffre d'affaires.

Écris une requête qui retourne pour chaque agent : son nom, le nombre de transactions, le CA total.

👉 Voir la solution

SQL

```
SELECT
  t.agent,
  COUNT(t.id_transaction) AS nb_transactions,
  SUM(t.montant)          AS chiffre_affaires
FROM transactions t
GROUP BY t.agent
ORDER BY chiffre_affaires DESC;
```

AGENT	NB_TRANSACTIONS	CHIFFRE_AFFAIRES
Kouassi Éric	4	6 000
Traoré Aminata	2	2 500
Bamba Seydou	2	3 000
N'Guessan Fatou	2	2 000
Koné Mamadou	3	4 000

📌 Ici, pas besoin de JOIN — toutes les infos agent sont dans `transactions`. JOIN n'est utile que quand les données sont dans des tables **différentes**.

Exercice B — Clients actifs vs inactifs

La direction commerciale veut deux listes : les clients qui ont acheté au moins une fois, et ceux qui n'ont encore rien acheté.

Écris deux requêtes séparées.

👉 Voir la solution

SQL

```
-- Clients avec au moins une transaction (INNER JOIN)
SELECT DISTINCT c.nom, c.ville
FROM clients c
JOIN transactions t ON c.id_client = t.id_client
ORDER BY c.nom;
```

SQL

```
-- Clients sans aucune transaction (anti-join)
SELECT c.nom, c.ville, c.date_inscription
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client
WHERE t.id_transaction IS NULL;
```



DISTINCT dans la première requête évite de voir Kouassi Ama 3 fois (une par transaction). On veut juste savoir si le client a acheté, pas combien de fois.

Exercice C — Vue enrichie complète

Crée une vue qui affiche pour chaque transaction : le nom du client, sa ville, le nom du forfait, son type et le montant payé.

👉 Voir la solution

SQL

```
SELECT
  c.nom          AS client,
  c.ville,
  f.nom_forfait,
  f.type_forfait,
  t.montant,
  t.mois
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
JOIN forfaits f ON t.id_forfait = f.id_forfait
ORDER BY t.id_transaction;
```

→ **12 lignes** — une par transaction, enrichie avec les infos client et forfait.

🎯 C'est exactement le genre de requête qu'un DA junior écrit tous les jours pour préparer un export vers Excel ou Power BI.

6. RIGHT JOIN — symétrique du LEFT JOIN

RIGHT JOIN fonctionne exactement comme **LEFT JOIN**, mais en gardant **toutes les lignes de la table de droite** (celle après **RIGHT JOIN**).

SQL

```
-- Ces deux requêtes sont équivalentes
SELECT * FROM clients c LEFT JOIN transactions t ON c.id_client =
t.id_client;
SELECT * FROM transactions t RIGHT JOIN clients c ON t.id_client =
c.id_client;
```

⚠ **En pratique, les Data Analysts n'utilisent presque jamais RIGHT JOIN.** Il suffit d'inverser l'ordre des tables et d'utiliser LEFT JOIN — c'est plus lisible et universellement compris.

Règle : chaque fois que tu es tenté d'écrire un RIGHT JOIN, retourne-le en LEFT JOIN.

SQL

```
-- ❌ RIGHT JOIN - peu lisible
SELECT c.nom, t.montant
FROM transactions t
RIGHT JOIN clients c ON t.id_client = c.id_client;

-- ✅ LEFT JOIN équivalent - préféré
SELECT c.nom, t.montant
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client;
```

SQLite supporte RIGHT JOIN depuis la version 3.39 (2022). Sur les anciennes versions, il peut provoquer une erreur — raison de plus pour lui préférer LEFT JOIN.

7. FULL OUTER JOIN — tout garder des deux côtés

Le principe

FULL OUTER JOIN retourne **toutes les lignes des deux tables**, qu'elles aient une correspondance ou non. Les colonnes sans correspondance sont **NULL** de chaque côté.

Table gauche		Table droite		FULL OUTER JOIN
1042	↔	1042	✅	1042 · données gauche + droite
7001	↔	???	❌	7001 · données gauche + NULL
???	↔	9999	❌	NULL · NULL + données droite

Cas d'usage typique : réconciliation de deux systèmes — trouver à la fois les clients sans transaction ET les transactions avec un `id_client` inconnu (données corrompues).

SQLite ne supporte pas FULL OUTER JOIN nativement

Si tu tapes `FULL OUTER JOIN` dans SQLite, tu obtiens une erreur. Mais on peut l'**émuler** avec `LEFT JOIN` + `UNION ALL` — et ça introduit une technique qu'on reverra en M09.

Émulation avec UNION ALL

Le principe :

1. **Partie 1** : `LEFT JOIN` → toutes les lignes de gauche + correspondances droite
2. **Partie 2** : `LEFT JOIN inversé` + `WHERE gauche IS NULL` → les lignes de droite qui n'ont pas de correspondance à gauche

```
FULL OUTER JOIN = LEFT JOIN (tout de gauche)
                  UNION ALL
                  LEFT JOIN inversé WHERE gauche IS NULL (orphelins de
droite)
```

Exemple — Audit complet : clients et transactions, orphelins inclus

SQL

```
-- Partie 1 : tous les clients (avec ou sans transaction)
SELECT
  c.nom      AS client,
  c.ville,
  t.montant,
  t.mois
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client

UNION ALL

-- Partie 2 : transactions sans client connu (données corrompues)
SELECT
  NULL      AS client,
  NULL      AS ville,
  t.montant,
  t.mois
FROM transactions t
LEFT JOIN clients c ON t.id_client = c.id_client
WHERE c.id_client IS NULL;
```

Ce que retourne cette requête :

- Toutes les transactions avec le nom du client ✓
- Les clients sans aucune transaction (montant = NULL) ✓
- Les transactions sans client connu dans notre dataset → 0 ici, mais en production c'est fréquent ✓



UNION ALL empile les résultats de deux requêtes l'une sous l'autre. Les deux **SELECT** doivent avoir le **même nombre de colonnes**. On l'explorera plus en détail en M09.

Quand utiliser FULL OUTER JOIN ?

SITUATION	JOIN RECOMMANDÉ
Je veux seulement les données communes	INNER JOIN
Je veux tout d'un côté + les correspondances	LEFT JOIN
Je veux tout des deux côtés, orphelins inclus	FULL OUTER JOIN (émulé)
Audit, réconciliation, données sales	FULL OUTER JOIN (émulé)

🔑 En PostgreSQL, BigQuery ou Snowflake, **FULL OUTER JOIN** est supporté nativement. La syntaxe est identique à **INNER JOIN** — seul le mot-clé change. Ce que tu viens d'apprendre ici te prépare à l'utiliser directement dans ces environnements.

8. JOIN + GROUP BY — la vraie puissance

Les jointures seules affichent des lignes. Combinées avec **GROUP BY**, elles répondent à des **questions business réelles** qu'une seule table ne peut pas résoudre.

Question 1 — Chiffre d'affaires par client

« Qui sont nos meilleurs clients ? »

SQL

```
SELECT
  c.nom,
  c.ville,
  COUNT(t.id_transaction) AS nb_achats,
  SUM(t.montant)          AS chiffre_affaires
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.id_client, c.nom, c.ville
ORDER BY chiffre_affaires DESC;
```

📌 Pourquoi `GROUP BY c.id_client, c.nom, c.ville` et pas juste `GROUP BY c.nom` ? On groupe par la clé primaire (`id_client`) pour éviter toute ambiguïté si deux clients portent le même nom. Les colonnes `c.nom` et `c.ville` sont ajoutées pour pouvoir les afficher dans le SELECT — elles sont fonctionnellement dépendantes de `id_client`.

NOM	VILLE	NB_ACHATS	CHIFFRE_AFFAIRES
Kouassi Ama	Abidjan	3	5 000
Traoré Bakary	Bouaké	2	2 500
Yao Serge	Abidjan	2	2 000
Ouédraogo Fatima	Yamoussoukro	1	2 000
Koné Mariama	San Pedro	2	2 000
Diallo Ibrahim	Abidjan	2	2 000

Question 2 — Chiffre d'affaires par type de forfait

« Quel type de produit génère le plus de revenus ? »

SQL

```
SELECT
  f.type_forfait,
  COUNT(t.id_transaction) AS nb_ventes,
  SUM(t.montant)          AS chiffre_affaires
FROM transactions t
JOIN forfaits f ON t.id_forfait = f.id_forfait
GROUP BY f.type_forfait
ORDER BY chiffre_affaires DESC;
```

TYPE_FORFAIT	NB_VENTES	CHIFFRE_AFFAIRES
Forfait	6	11 000
Recharge	6	4 500

Question 3 — Catalogue complet des forfaits avec leurs performances

« Pour chaque forfait du catalogue, combien de ventes et quel CA — y compris les forfaits jamais vendus ? »

SQL

```
SELECT
  f.nom_forfait,
  f.prix,
  f.type_forfait,
  COUNT(t.id_transaction) AS nb_ventes,
  COALESCE(SUM(t.montant), 0) AS chiffre_affaires
FROM forfaits f
LEFT JOIN transactions t ON f.id_forfait = t.id_forfait
GROUP BY f.id_forfait, f.nom_forfait, f.prix, f.type_forfait
ORDER BY nb_ventes DESC;
```

NOM_FORFAIT	PRIX	TYPE_FORFAIT	NB_VENTES	CHIFFRE_AFFAIRES
Forfait_internet	2 000	Forfait	4	8 000
Recharge_500	500	Recharge	3	1 500
Recharge_1000	1 000	Recharge	3	3 000
Forfait_voix	1 500	Forfait	2	3 000
Forfait_data_nuit	800	Forfait	0	0
Pack_famille	3 500	Pack	0	0



COALESCE(valeur, 0) remplace les **NULL** par **0**. Sans ça, **SUM(NULL)** retourne **NULL** — moins lisible dans un rapport. C'est une bonne pratique systématique avec LEFT JOIN + agrégation.

9. Les pièges classiques

Piège 1 — Ambiguïté de colonne : toujours préfixer avec l'alias

SQL

```
-- ❌ ERREUR : id_client existe dans les deux tables – SQL ne sait pas laquelle
```

```
SELECT id_client, nom, montant
FROM transactions
JOIN clients ON transactions.id_client = clients.id_client;
```

```
-- ✅ Correct : alias partout
```

```
SELECT t.id_client, c.nom, t.montant
FROM transactions t
JOIN clients c ON t.id_client = c.id_client;
```

Piège 2 — Oublier le ON : le produit cartésien

SQL

```
-- ❌ CATASTROPHE : JOIN sans ON
```

```
SELECT t.montant, c.nom
FROM transactions t
JOIN clients c;
```

```
-- Résultat : 12 transactions × 8 clients = 96 lignes absurdes
```

Sans **ON**, SQL crée toutes les combinaisons possibles entre les deux tables. Avec des grandes tables (1M × 1M), ça peut faire planter le serveur.

Piège 3 — Joindre sur la mauvaise clé

SQL

```
-- ❌ FAUX – jointure sur montant = prix (coïncidence numérique, pas logique)
```

```
SELECT t.montant, f.nom_forfait
FROM transactions t
JOIN forfaits f ON t.montant = f.prix;
```

```
-- ✅ Correct – jointure sur l'identifiant métier
```

```
SELECT t.montant, f.nom_forfait
FROM transactions t
JOIN forfaits f ON t.id_forfait = f.id_forfait;
```

Toujours joindre sur des **clés primaires / étrangères**, pas sur des valeurs numériques qui pourraient coïncider par hasard.

Piège 4 — Doublons inattendus

Si une table a plusieurs lignes pour la même clé (données non normalisées), le JOIN multiplie les lignes.

```
clients : id=1042 apparaît 1 fois
transactions : id_client=1042 apparaît 3 fois
→ JOIN retourne 3 lignes pour ce client ✅ (c'est normal ici)
```

Mais si `clients` avait **2 lignes** pour id=1042 (doublon) :

```
→ JOIN retourne 3 × 2 = 6 lignes pour ce client ❌
```

Réflexe : si ton résultat semble trop grand, vérifie la cardinalité de chaque table avec `SELECT COUNT(*)` et `COUNT(DISTINCT cle)`.

Réflexe de débogage

Si ton résultat JOIN semble bizarre (trop de lignes, doublons inattendus) :

```
SQL
-- 1. Vérifie le nombre de lignes de chaque table
SELECT COUNT(*) FROM clients;      -- attendu : 8
SELECT COUNT(*) FROM transactions; -- attendu : 12

-- 2. Aperçu rapide du résultat avant d'aller plus loin
SELECT t.id_client, c.nom, t.montant
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
LIMIT 5;
```

💡 **Règle** : commence toujours avec `LIMIT 5` quand tu construis une nouvelle requête JOIN. Vérifie que les colonnes contiennent ce que tu attends avant d'ajouter `GROUP BY` ou `ORDER BY`.

10. Cheatsheet — Jointures

Les 4 jointures en un coup d'œil

```
INNER JOIN      : lignes communes aux deux tables
LEFT JOIN       : tout de gauche + NULL si pas de match à droite
RIGHT JOIN      : tout de droite + NULL (→ réécrire en LEFT JOIN)
FULL OUTER JOIN : tout des deux côtés (émulé avec UNION ALL en SQLite)
```

Syntaxe complète

```
SQL
SELECT t.colonne, c.colonne, f.colonne
FROM transactions t
JOIN      clients c ON t.id_client = c.id_client  -- INNER (défaut)
JOIN      forfaits f ON t.id_forfait = f.id_forfait;

SELECT c.nom, t.montant
FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client;
```

Anti-join — orphelins d'un côté

```
SQL
-- Clients sans transaction
SELECT c.nom FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client
WHERE t.id_transaction IS NULL;
```

FULL OUTER JOIN émulé (SQLite)

```
SQL
SELECT c.nom, t.montant FROM clients c
LEFT JOIN transactions t ON c.id_client = t.id_client
UNION ALL
SELECT NULL, t.montant FROM transactions t
LEFT JOIN clients c ON t.id_client = c.id_client
WHERE c.id_client IS NULL;
```

JOIN + GROUP BY

```
SQL
SELECT c.nom, SUM(t.montant) AS ca
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.id_client, c.nom
ORDER BY ca DESC;
```

COALESCE — remplacer les NULL par 0

```
SQL
COALESCE(SUM(t.montant), 0) -- retourne 0 si SUM est NULL
```

Ordre d'exécution avec JOIN

FROM + JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

11. Quiz — Vérifie ta compréhension

Q1. Tu as une table `commandes` et une table `produits`. La table `commandes` a 500 lignes et `produits` a 50 lignes. Tu fais un `INNER JOIN` et obtiens 480 lignes. Qu'est-ce que ça signifie ?

- a) Il y a une erreur dans la requête
- b) 20 commandes font référence à des produits qui n'existent pas dans la table `produits`
- c) La table `produits` est incomplète

 [Voir la réponse](#)

 **b)** — `INNER JOIN` ne retourne que les lignes qui matchent dans les deux tables. $500 - 480 = 20$ commandes ont un `id_produit` qui ne correspond à aucune entrée dans `produits`. Ce sont des données corrompues ou orphelines.

Q2. Quelle est la différence entre `LEFT JOIN` et `INNER JOIN` ?

- a) `LEFT JOIN` est plus rapide qu'`INNER JOIN`
- b) `LEFT JOIN` garde toutes les lignes de la table de gauche même sans correspondance, `INNER JOIN` non
- c) `LEFT JOIN` ne fonctionne qu'avec deux tables

 [Voir la réponse](#)

 **b)** — `INNER JOIN` supprime les lignes sans correspondance des deux côtés. `LEFT JOIN` conserve toutes les lignes de la table de gauche et place `NULL` dans les colonnes de droite quand il n'y a pas de correspondance.

Q3. Tu veux trouver les produits qui n'ont jamais été commandés. Quelle est la bonne approche ?

- a) `INNER JOIN` entre `produits` et `commandes`
- b) `LEFT JOIN` depuis `produits` vers `commandes` + `WHERE commandes.id IS NULL`
- c) `SELECT * FROM produits WHERE id NOT IN (SELECT id FROM commandes)`

👉 Voir la réponse

✅ **b) ET c) sont tous les deux corrects**, mais b) (anti-join avec `LEFT JOIN`) est la méthode recommandée en SQL professionnel car elle est plus performante sur les grandes tables. Le `NOT IN` pose des problèmes avec les valeurs `NULL`.

Q4. Cette requête a un problème. Lequel ?

```
SQL
SELECT id_client, nom, montant
FROM transactions t
JOIN clients c ON t.id_client = c.id_client;
```

- a) Il manque un `GROUP BY`
- b) `id_client` est ambigu — il existe dans les deux tables
- c) On ne peut pas sélectionner `nom` depuis la table `clients`

👉 Voir la réponse

✅ **b) —** `id_client` existe dans `transactions` ET dans `clients`. SQL ne sait pas laquelle utiliser → erreur d'ambiguïté. Solution : `t.id_client` ou `c.id_client`.

Q5. Que se passe-t-il si tu fais un `JOIN` sans clause `ON` ?

- a) SQL retourne une erreur de syntaxe
- b) SQL fait un produit cartésien (toutes les combinaisons possibles)
- c) SQL fait automatiquement un `INNER JOIN` sur les colonnes de même nom

👉 Voir la réponse

✅ b) — Sans `ON`, SQL crée le produit cartésien : chaque ligne de la table gauche est combinée avec chaque ligne de la table droite. Avec $1\,000 \times 1\,000$ lignes, tu obtiens 1 000 000 de lignes. C'est souvent une erreur accidentelle.

Q6. Tu veux afficher tous les forfaits du catalogue avec le nombre de fois qu'ils ont été vendus, y compris les forfaits jamais vendus (nb = 0). Quel type de JOIN utiliser ?

- a) `INNER JOIN forfaits f ON t.id_forfait = f.id_forfait`
- b) `LEFT JOIN transactions t ON f.id_forfait = t.id_forfait` depuis `forfaits`
- c) `RIGHT JOIN forfaits f ON t.id_forfait = f.id_forfait` depuis `transactions`

👉 Voir la réponse

✅ b) — On part de `forfaits` (la table "maître") et on fait un LEFT JOIN vers `transactions`. Les forfaits sans transaction auront NULL dans les colonnes de `transactions`. Avec `COUNT(t.id_transaction)`, les forfaits jamais vendus retournent 0.

Q7. Pourquoi SQLite ne supporte-t-il pas `FULL OUTER JOIN` nativement, et comment l'émuler ?

- a) SQLite est trop ancien pour cette fonctionnalité — impossible à contourner
- b) On émule avec `LEFT JOIN UNION ALL LEFT JOIN inversé WHERE IS NULL`
- c) On émule avec deux `INNER JOIN` combinés

👉 Voir la réponse

✅ b) — L'émulation standard est :

Partie 1 : `A LEFT JOIN B` → tout de A avec correspondances de B

Partie 2 : `B LEFT JOIN A WHERE A.key IS NULL` → les lignes de B sans correspondance dans A
Les deux parties sont combinées avec `UNION ALL`.

Q8. Tu veux le CA total par ville de client. Quelle requête est correcte ?

SQL

-- Option A

```
SELECT t.region, SUM(t.montant) FROM transactions t GROUP BY t.region;
```

-- Option B

```
SELECT c.ville, SUM(t.montant) AS ca
FROM transactions t
JOIN clients c ON t.id_client = c.id_client
GROUP BY c.ville
ORDER BY ca DESC;
```

👉 Voir la réponse

✅ **Option B** — `t.region` est la région de l'agent qui a vendu, pas la ville du client. Pour obtenir la ville du client, il faut joindre la table `clients`. Cette distinction (ville client ≠ région agent) est un exemple typique d'erreur d'analyse qu'un JOIN bien pensé évite.

Q9. `COALESCE(SUM(t.montant), 0)` — à quoi sert cette fonction dans un LEFT JOIN ?

- a) Elle arrondit la somme à 0 décimale
- b) Elle remplace NULL par 0 quand la somme n'a pas de valeur (aucune transaction)
- c) Elle ignore les valeurs NULL dans le calcul de la somme

👉 Voir la réponse

✅ **b)** — Quand un LEFT JOIN ne trouve pas de transaction pour un forfait, `SUM(t.montant)` retourne `NULL` (pas 0). `COALESCE(SUM(t.montant), 0)` remplace ce NULL par 0, ce qui est bien plus lisible dans un rapport.

Q10. Pourquoi préfère-t-on `LEFT JOIN` à `RIGHT JOIN` en pratique ?

- a) `LEFT JOIN` est plus rapide
- b) `RIGHT JOIN` ne fonctionne pas dans tous les moteurs SQL

- c) Tout **RIGHT JOIN** peut être réécrit en **LEFT JOIN** en inversant l'ordre des tables — c'est plus lisible et standard

👉 Voir la réponse

✅ c) — La convention universelle est d'utiliser LEFT JOIN et de mettre la table "maître" (celle dont on veut garder toutes les lignes) à gauche, après FROM. Cela rend le code plus lisible et prévisible. Le b) est partiellement vrai (SQLite l'a supporté tardivement), mais la vraie raison est la convention de lisibilité.

12. Résumé du module

CONCEPT	CE QU'IL FAIT	QUAND L'UTILISER
INNER JOIN	Lignes avec correspondance dans les deux tables	Données propres, on veut seulement les matches
LEFT JOIN	Tout de gauche + NULL si pas de match	Garder tous les enregistrements d'une table maître
Anti-join	LEFT JOIN + WHERE droite IS NULL	Trouver les orphelins (clients sans commande, etc.)
RIGHT JOIN	Tout de droite + NULL	Éviter — réécrire en LEFT JOIN
FULL OUTER JOIN	Tout des deux côtés	Audit, réconciliation (émulé avec UNION ALL en SQLite)
Alias de table	t , c , f	Toujours avec JOIN pour éviter l'ambiguïté
ON	Condition de jointure	Obligatoire — sans ON = produit cartésien
COALESCE(val, 0)	Remplace NULL par 0	Avec LEFT JOIN + agrégation
JOIN + GROUP BY	Agrégation multi-table	Questions business réelles

→ Module suivant

Tu sais maintenant joindre des tables et combiner les données. Dans le **Module 09**, on monte en puissance avec les outils avancés du reporting SQL :

- **Sous-requêtes** — une requête dans une requête
- **CTEs** (**WITH**) — nommer et réutiliser des résultats intermédiaires
- **CASE WHEN** — logique conditionnelle dans SQL
- **Window functions** — **RANK()**, **ROW_NUMBER()**, **LAG()**
- **L'IA pour écrire du SQL** — utiliser Claude ou ChatGPT efficacement

→ Module 09 — SQL reporting & IA